SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I __Chalamala Sai Harshitha (192424346)__ *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C. Sai Harshitha

1. **A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.**

Introduction

A perceptron is a simple neural network used for binary classification. It is designed to classify linearly separable data. According to the perceptron convergence theorem, it should converge after a finite number of iterations. If it does not converge, the problem is usually caused by incorrect parameter settings, implementation errors, or data-related issues.

Possible Causes

1. Improper Learning Rate

The learning rate determines how much the weights are updated during training.

- A **high learning rate** causes large weight changes, making the model unstable.
- A **low learning rate** results in very slow learning and requires many iterations.

2. Incorrect Weight Initialization

Weights should be initialized with small random values.

- Large or improper initial weights may delay convergence and reduce learning efficiency.

3. Bias Handling Errors

The bias shifts the decision boundary to improve classification.

- If the bias is missing or updated incorrectly, the perceptron may fail to classify the data correctly.

4. Incorrect Weight Update Rule

The perceptron updates its weights only when a sample is misclassified.

- Errors in the update equation or programming mistakes can prevent convergence.

5. Data Issues

Although the dataset is linearly separable, problems such as incorrect class labels, noisy data, or unnormalized features can affect the learning process.

Debugging Strategies

1. Verify that the dataset is truly linearly separable and free from labeling errors.
2. Check whether the learning rate is suitable and adjust it if necessary.
3. Monitor the weight and bias values after each iteration.
4. Validate the implementation of the weight update rule.
5. Track the number of misclassified samples to ensure that the training error decreases over time.

Optimization Techniques

- Use an appropriate learning rate for stable convergence.
- Initialize weights with small random values.
- Normalize or standardize the input features before training.
- Update both weights and bias correctly during every misclassification.
- Stop training when all samples are classified correctly or when no further improvement is observed.

Result

By selecting an appropriate learning rate, properly initializing weights, correctly updating the bias and weight values, and normalizing the input data, the perceptron converges faster and achieves stable and accurate classification for linearly separable datasets.

2. **A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.**

Introduction

A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks. It divides the dataset into smaller subsets based on feature values to make predictions. If the model performs well on the training data but poorly on unseen data, it indicates overfitting, where the model learns the training data too closely and fails to generalize.

Possible Causes

1. Overfitting

The decision tree memorizes the training data, including noise and unnecessary details, resulting in poor performance on new data.

2. Deep Tree Structure

A tree with too many levels creates highly specific rules that fit only the training dataset instead of general patterns.

3. Noisy Data

Incorrect, missing, or inconsistent data can produce misleading decision rules and reduce prediction accuracy.

4. Irrelevant Features

Including unnecessary features increases model complexity and may lead to incorrect splits during training.

5. Insufficient Training Data

A small or unrepresentative dataset may not capture the actual data distribution, affecting model performance.

Debugging Strategies

1. Compare training accuracy and testing accuracy to identify overfitting.
2. Visualize the decision tree and check whether it has unnecessary depth.
3. Examine the dataset for missing values, duplicate records, and noisy data.

4. Evaluate the importance of each feature and remove irrelevant ones.
5. Use validation data to check whether the model generalizes well.

Optimization Techniques

- **Pruning:** Remove unnecessary branches to reduce model complexity and prevent overfitting.
- **Cross-Validation:** Evaluate the model on different data splits to improve reliability.
- **Feature Selection:** Keep only important features for training.
- **Limit Tree Depth:** Set a maximum depth to avoid overly complex trees.
- **Data Cleaning:** Remove noise, handle missing values, and eliminate duplicate records before training.

Result

By applying pruning, cross-validation, feature selection, limiting the tree depth, and cleaning the dataset, the Decision Tree becomes less prone to overfitting. These optimization techniques improve the model's ability to generalize, resulting in better accuracy on unseen data while maintaining reliable performance.

3. **A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.**

Introduction

K-Means is an unsupervised machine learning algorithm used to group similar data points into **K clusters** based on their similarity. It assigns each data point to the nearest centroid and updates the centroids until convergence. If the clusters are highly imbalanced and do not represent the actual data distribution, it is usually due to incorrect parameter selection, poor centroid initialization, or data-related issues.

Possible Causes

1. Incorrect Choice of K

Choosing too many or too few clusters results in poor grouping and does not accurately represent the data distribution.

2. Poor Centroid Initialization

Randomly selecting initial centroids may lead to poor cluster formation and inconsistent results across different runs.

3. Presence of Outliers

Outliers can significantly influence the centroid positions, resulting in imbalanced or incorrect clusters.

4. Unnormalized Data

Features with different scales can dominate the distance calculation, causing inaccurate cluster assignments.

5. Uneven Data Distribution

If the dataset contains clusters of different sizes or densities, K-Means may fail to identify them correctly.

Debugging Strategies

1. Test different values of **K** and compare the clustering results.
2. Visualize the clusters and centroid positions to identify incorrect grouping.
3. Check the dataset for outliers and remove or treat them appropriately.

4. Normalize the input features before clustering.
5. Run the algorithm multiple times to verify the consistency of the cluster assignments.

Optimization Techniques

- **Use the Elbow Method:** Select the optimal value of **K** by analyzing the within-cluster sum of squares.
- **Apply K-Means++:** Initialize centroids more effectively to improve clustering accuracy.
- **Remove Outliers:** Eliminate extreme values that distort centroid locations.
- **Normalize Features:** Scale all features to a similar range before clustering.
- **Perform Multiple Initializations:** Run K-Means several times and choose the solution with the lowest clustering error.

Result

By selecting an appropriate value of **K**, using **K-Means++** for better centroid initialization, removing outliers, and normalizing the data, K-Means produces balanced and meaningful clusters. These optimization techniques improve clustering accuracy, stability, and better represent the actual data distribution.

4. **A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.**

Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression. It works by finding the optimal hyperplane that separates different classes. When applied to a **nonlinear dataset**, the model may underperform if the kernel, parameters, or input features are not chosen correctly.

Possible Causes

1. Incorrect Kernel Choice

A linear kernel may not capture the complex patterns present in nonlinear data, resulting in poor classification performance.

2. Improper Parameter Tuning

Incorrect values of **C** (regularization parameter) and **gamma** (kernel parameter) can lead to overfitting or underfitting.

3. Feature Scaling Problems

SVM is sensitive to feature values. If features are not normalized, variables with larger values dominate the decision boundary.

4. Noisy or Imbalanced Data

Noise, missing values, or class imbalance can reduce the model's ability to classify data accurately.

5. Insufficient Feature Representation

The selected features may not contain enough information to separate the classes effectively.

Debugging Strategies

1. Check whether the selected kernel is suitable for the dataset.
2. Compare the performance of different kernels such as Linear, Polynomial, and RBF.
3. Normalize or standardize the input features before training.
4. Test different values of **C** and **gamma** to identify the best combination.

5. Evaluate the model using validation data and analyze the confusion matrix or accuracy scores.

Optimization Techniques

- **Kernel Selection:** Choose an appropriate kernel (such as RBF) for nonlinear datasets.
- **Hyperparameter Tuning:** Optimize **C** and **gamma** using Grid Search or Cross-Validation.
- **Feature Scaling:** Normalize or standardize all input features.
- **Feature Selection:** Remove irrelevant features to improve classification performance.
- **Data Preprocessing:** Handle missing values, remove noise, and balance the dataset before training.

Result

By selecting the appropriate kernel, tuning the **C** and **gamma** parameters, scaling the input features, and preprocessing the dataset, the SVM model can accurately classify nonlinear data. These optimization techniques improve generalization, increase prediction accuracy, and produce a more reliable classification model.

5. **A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods.**

Introduction

Naïve Bayes is a supervised machine learning algorithm commonly used for **text classification**, such as spam detection and sentiment analysis. It is based on Bayes' theorem and assumes that all features are independent of each other. When applied to real-world data, its performance may decrease due to noisy data, irrelevant features, or violation of the independence assumption.

Possible Causes

1. Violation of Feature Independence

Naïve Bayes assumes that all features are independent. In real-world text data, many words are related to each other, reducing the model's accuracy.

2. Improper Preprocessing

If stop words, punctuation, special characters, or unnecessary words are not removed, they introduce noise and affect classification performance.

3. Imbalanced Dataset

If one class contains significantly more samples than another, the classifier may become biased toward the majority class.

4. Noisy and Irrelevant Features

Unnecessary words and irrelevant features increase model complexity and reduce prediction accuracy.

5. Zero Probability Problem

If a word does not appear in the training data for a particular class, the probability becomes zero, leading to incorrect predictions.

Debugging Strategies

1. Check whether the text data has been properly preprocessed by removing stop words, punctuation, and unnecessary symbols.

2. Analyze the dataset to identify class imbalance and noisy features.
3. Evaluate the importance of features and remove irrelevant words.
4. Verify whether feature independence assumptions are significantly violated.
5. Test the classifier using validation data and evaluate accuracy, precision, and recall.

Optimization Techniques

- **Feature Selection:** Remove irrelevant and less informative words.
- **Smoothing Techniques:** Apply Laplace (Add-One) smoothing to avoid zero probability problems.
- **Data Balancing:** Use oversampling or undersampling techniques to balance class distribution.
- **Text Preprocessing:** Perform tokenization, stop-word removal, and stemming or lemmatization.
- **Cross-Validation:** Validate the model on different data splits to improve generalization.

Result

By applying proper text preprocessing, selecting relevant features, using Laplace smoothing, balancing the dataset, and validating the model through cross-validation, the Naïve Bayes classifier achieves better accuracy, improved generalization, and more reliable text classification results.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand